



UNIVERSITY
OF TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Corso di Laurea in Informatica

Maintaining Hitting Sets in Temporal Hypergraphs

Supervisore

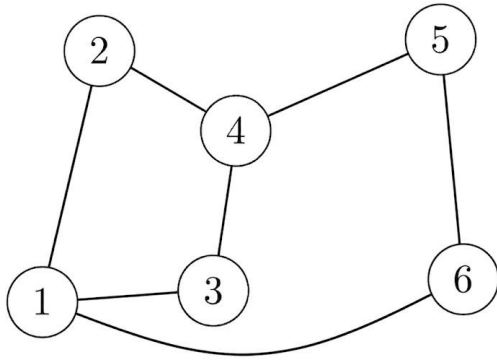
Alberto Montresor
Quintino Francesco Lotito

Laureando

Luca Prigione

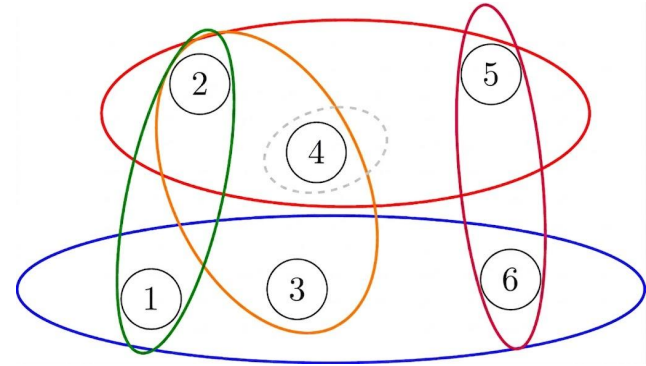
Grafi e ipergrafi

Grafi



- Archi uno-a-uno
- Relazioni binarie
- Struttura classica

Ipergrafi



- Iperarchi uno-a-molti e multi-a-molti
- Relazioni complesse
- Maggiore espressività

Limiti dei grafi classici

Comunicazioni originali

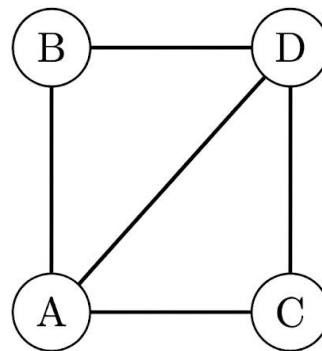
Chat 1 = {A, B, D}

Chat 2 = {A, B}

Chat 3 = {A, C}

Chat 4 = {C, D}

Rappresentazione tramite grafo



- Le comunicazioni di gruppo vengono trasformate in connessioni binarie
- Dal solo grafo non è possibile ricostruire univocamente le comunicazioni originali
- La struttura delle interazioni viene persa

Limiti dei grafi classici

Comunicazioni originali

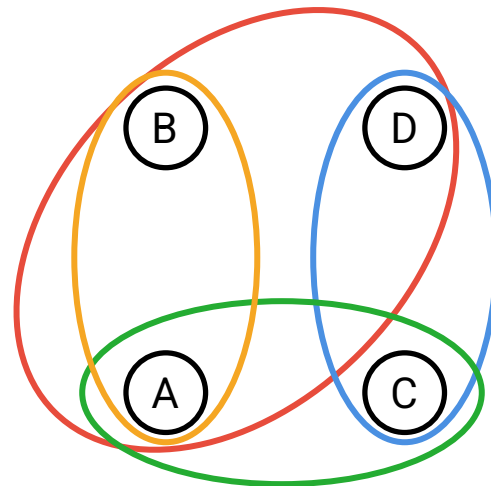
Chat 1 = {A, B, D}

Chat 2 = {A, B}

Chat 3 = {A, C}

Chat 4 = {C, D}

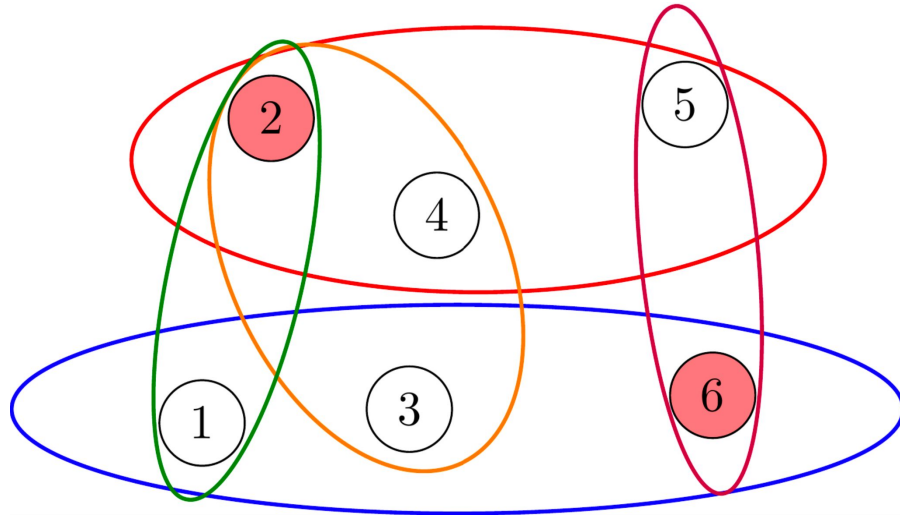
Rappresentazione tramite ipergrafo



Sistemi reali con interazioni di gruppo sono:

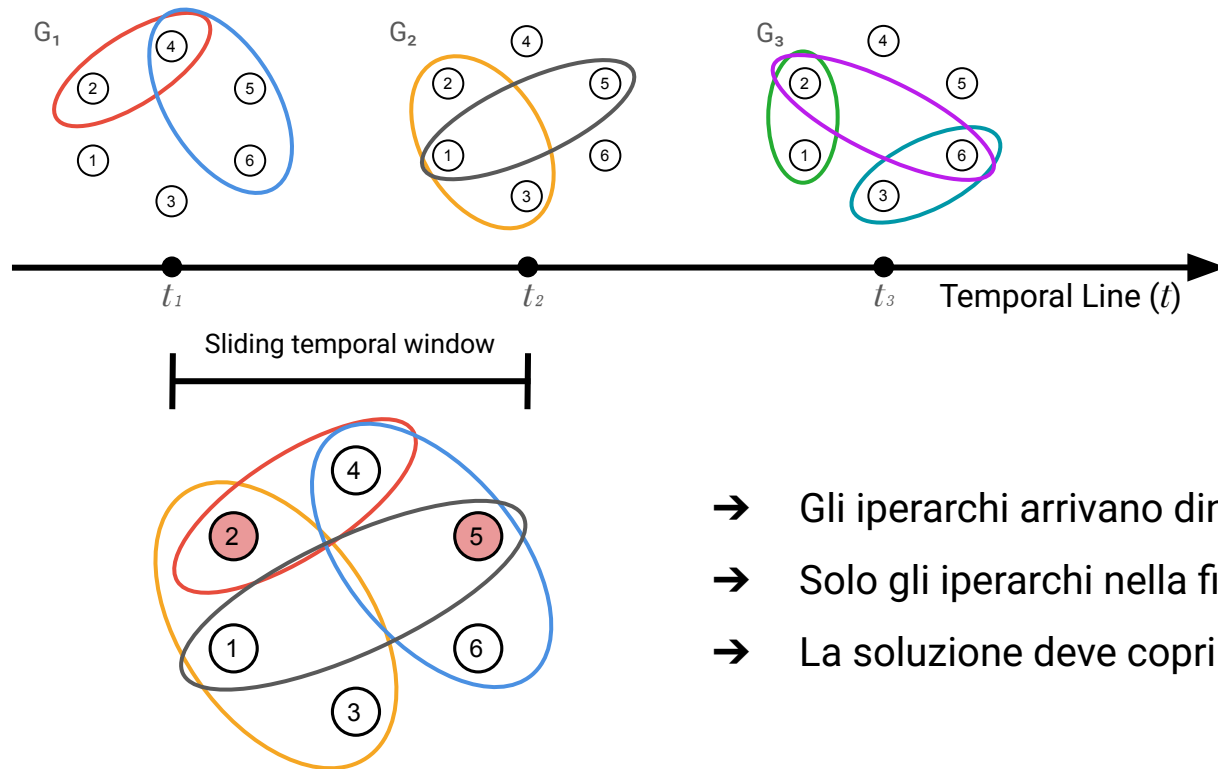
- | | |
|--|---|
| → Riunioni aziendali / videoconferenze | → Team di sviluppo software (commit/repository condivise) |
| → Gruppi WhatsApp / Telegram / Discord | → Social network (community) |
| → Collaborazioni scientifiche (paper con più autori) | → Blockchain e transazioni multi-input/multi-output |
| | → Sistemi peer-to-peer |

Cos'è un Hitting Set?



- Ogni iperarco deve contenere almeno un nodo appartenente all'Hitting Set
- Un singolo nodo può coprire più iperarchi contemporaneamente
- L'obiettivo è spesso quello di minimizzare il numero di nodi selezionati

Sliding Window Temporal Hitting Set



- Gli iperarchi arrivano dinamicamente nel tempo
- Solo gli iperarchi nella finestra corrente sono attivi
- La soluzione deve coprire tutte le richieste attive

Obiettivi del problema

Efficienza Computazionale



Aggiornare rapidamente la
soluzione

$$\text{Min } |T_{\text{update}}(t)|$$

Dimensione della soluzione



Minimizzare il numero di
nodi selezionati

$$\text{Min } |x(t)|$$

Riutilizzo dei nodi



Ridurre il numero complessivo di
nodi selezionati

$$\text{Min } \left| \bigcup_{t=1}^T x(t) \right|$$

Aspettative sperimentali

Window Shift Ridotto

- Prestazioni migliori quando lo shift è piccolo rispetto alla finestra
- Minori modifiche tra istanti consecutivi

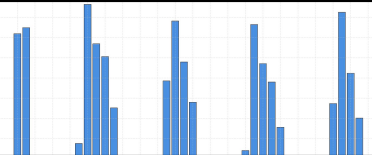
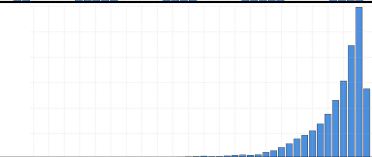
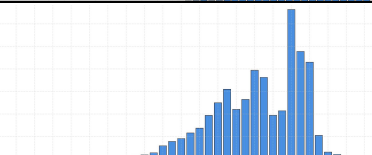
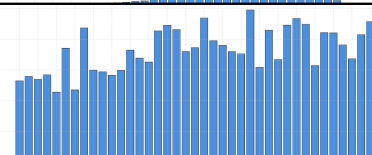
Maggiore Stabilità

- Aggiornare il meno possibile la soluzione corrente
- Riduzione delle variazioni tra hitting set consecutivi

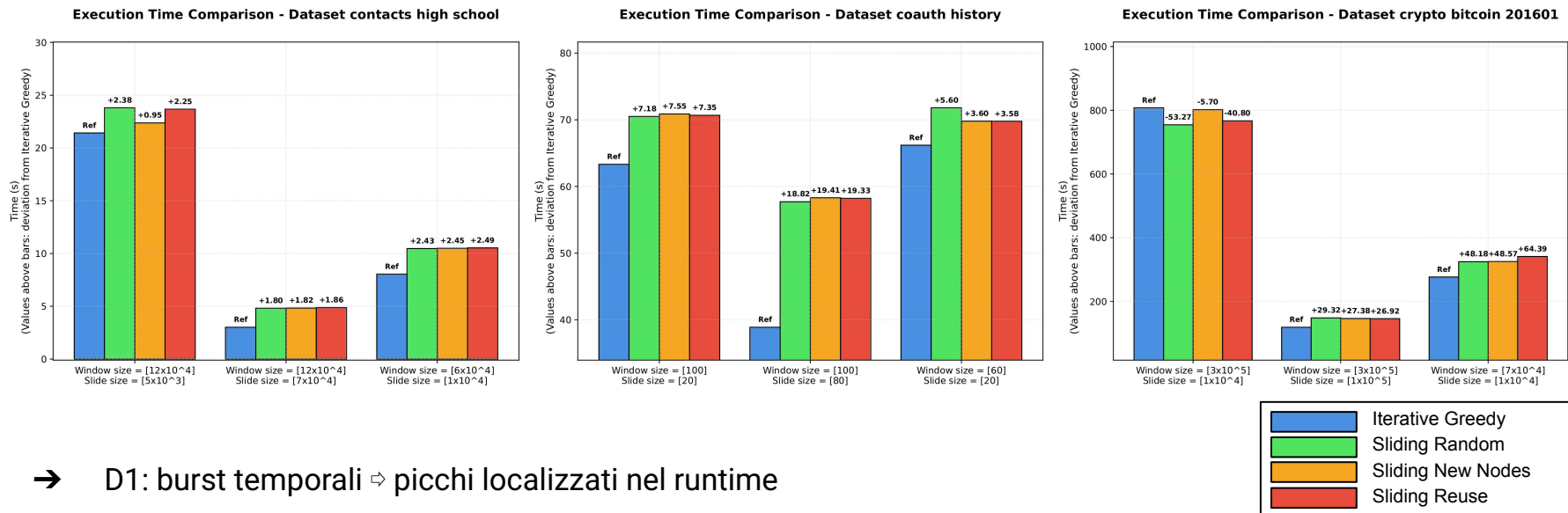
Aggiornamenti Veloci

- Elaborazione di un numero ridotto di iperarchi
- Migliori performance rispetto ai greedy classici

Caratteristiche dei Dataset

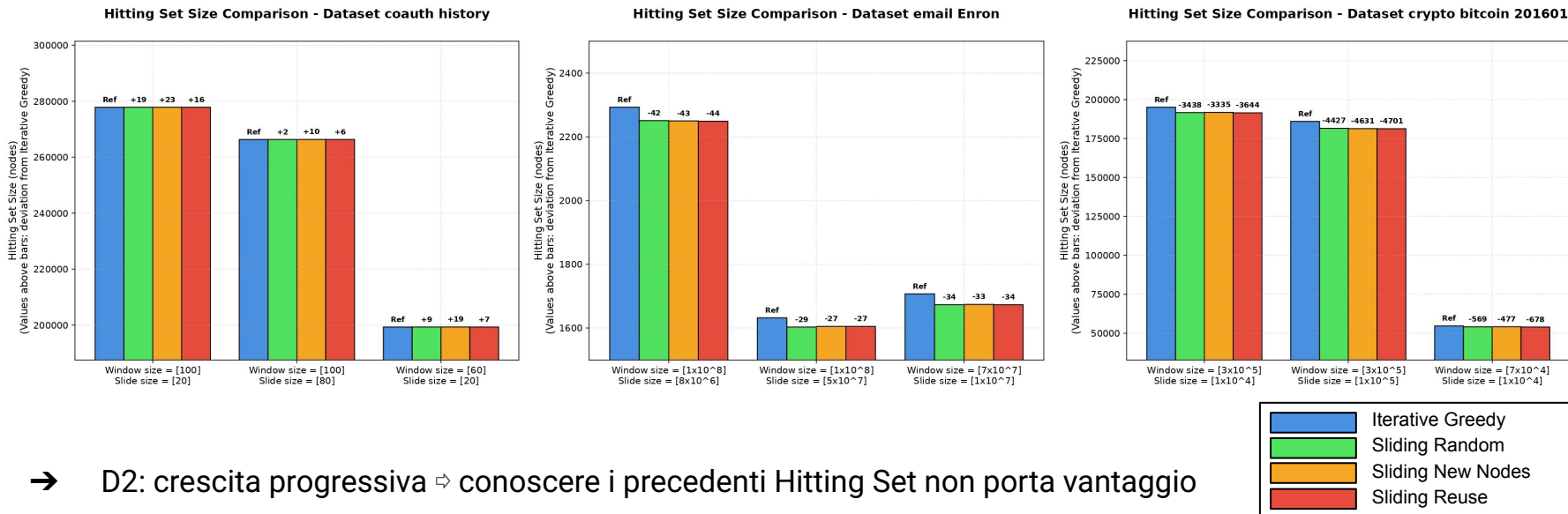
Dataset	Dimensione	Caratteristica distintiva	Pattern temporale
Contacts high school	Piccolo (3×10^2 Nodi)	Arrivi temporali a burst separati da periodi di inattività	
Coauth history	Grande (1×10^6 Nodi)	Distribuzione temporale esponenziale degli archi	
Email Enron	Medio ($8,4 \times 10^4$ Nodi)	Elevata densità di interazioni	
Crypto bitcoin	Grande ($2,5 \times 10^6$ Nodi)	Distribuzione temporale quasi costante	

Impatto della finestra temporale sul tempo di esecuzione totale



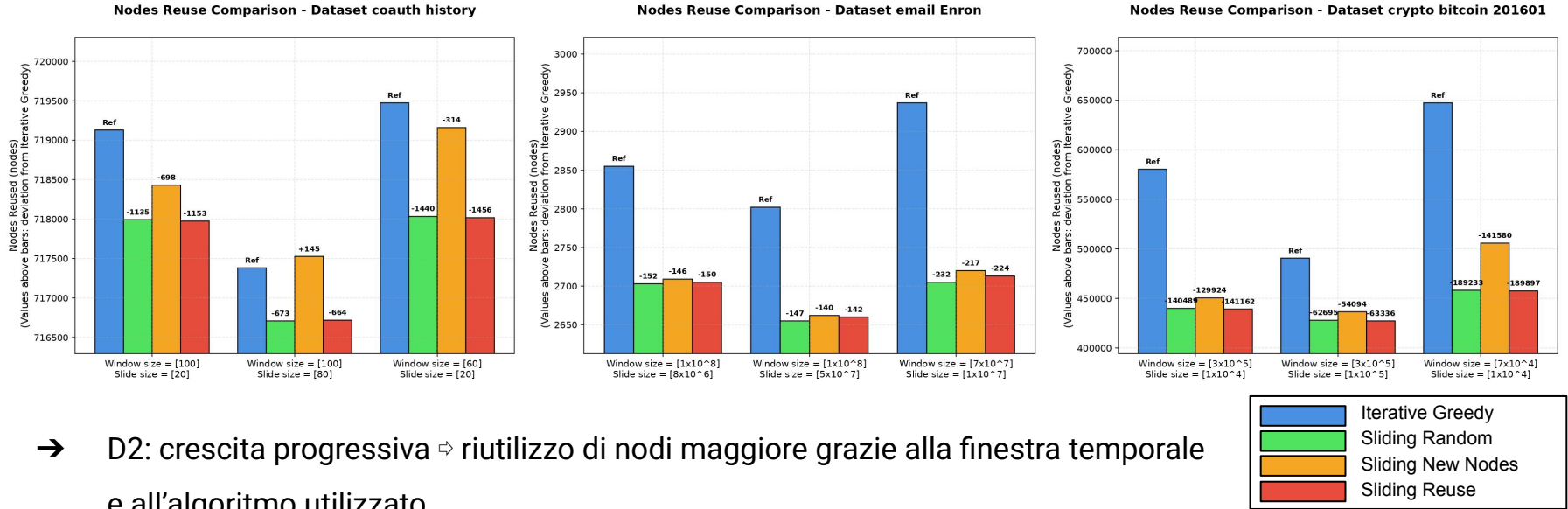
- D1: burst temporali ⇨ picchi localizzati nel runtime
- D2: crescita progressiva ⇨ aumento del costo degli aggiornamenti
- D4: distribuzione uniforme ⇨ maggiore riutilizzo della soluzione

Impatto della finestra temporale sul numero di nodi selezionati



- D2: crescita progressiva ⇨ conoscere i precedenti Hitting Set non porta vantaggio
- D3: elevata densità ⇨ a prescindere dall'algoritmo molti nodi vengono riutilizzati
- D4: distribuzione uniforme ⇨ vantaggio maggiore con la finestra temporale

Impatto della finestra temporale sul riutilizzo dei nodi



- D2: crescita progressiva ⇨ riutilizzo di nodi maggiore grazie alla finestra temporale e all'algoritmo utilizzato
- D3: elevata densità ⇨ riutilizzo di nodi maggiore grazie alla finestra temporale
- D4: distribuzione uniforme ⇨ riutilizzo di nodi maggiore grazie alla finestra temporale e in alcuni casi all'algoritmo utilizzato

Conclusioni

- L'approccio Sliding Window consente di mantenere un Hitting Set su ipergrafi temporali evitando di ricalcolare la soluzione da zero ad ogni aggiornamento.
- Le prestazioni dipendono fortemente dalle caratteristiche temporali del dataset e dalla dimensione della finestra selezionata.
- Nei dataset con elevato riutilizzo dei nodi, gli algoritmi basati sulla soluzione precedente ottengono significativi vantaggi in termini di tempo di esecuzione.
- Nei dataset molto densi o con forte crescita progressiva degli iperarchi, il beneficio del riutilizzo tende a ridursi.
- L'utilizzo di una finestra temporale permette in molti casi di aumentare il riutilizzo dei nodi e ridurre il costo degli aggiornamenti.
- I risultati mostrano che strategie incrementalmente possono rappresentare un'alternativa efficace rispetto al ricalcolo completo della soluzione.